

Codificação Para Front-end

16

2026

SENAI - CRTI

Revisão JavaScript e TypeScript

Fundamentos para tornar o e-commerce dinâmico

SENAI • DESENVOLVIMENTO FRONT-END



Onde Estamos no Projeto

O que já construímos

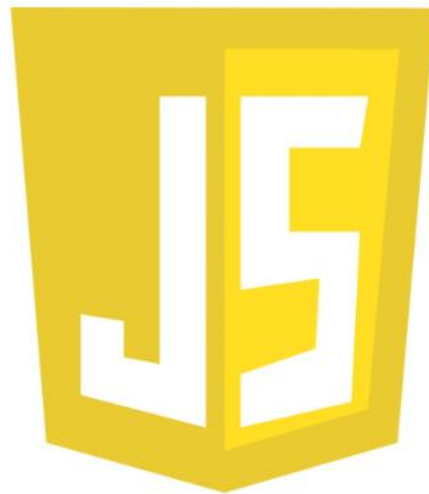
Nas aulas anteriores, desenvolvemos a **interface visual completa** do nosso e-commerce: estrutura HTML semântica, estilização com CSS, layout responsivo e componentes visuais como cards de produtos, cabeçalho e rodapé.

O projeto tem ótima aparência — mas ainda é estático. Nenhum dado é processado, nenhuma interação acontece de verdade.

O que faremos agora

- Adicionar lógica com JavaScript
- Manipular dados de produtos
- Preparar base para o DOM

JavaScript



O que é JavaScript e Seu Papel

A linguagem da web

JavaScript é a linguagem de programação nativa dos navegadores. Enquanto o **HTML** define a estrutura e o **CSS** cuida da aparência, o **JavaScript** é responsável pelo *comportamento* — tudo que acontece quando o usuário interage com a página.

No nosso e-commerce

- Calcular preços com desconto em tempo real
- Listar produtos dinamicamente
- Validar formulários de cadastro
- Reagir a cliques do usuário
- Comunicar-se com APIs de produtos

i JavaScript roda diretamente no navegador — não precisa de servidor para executar a lógica básica que veremos hoje.

Situação de Aprendizagem

Contexto

O e-commerce já possui interface pronta, mas ainda não possui comportamento dinâmico.

Desafio

Sua tarefa é iniciar a implementação de lógica com JavaScript para manipular dados e preparar o sistema para funcionalidades reais.

Entregável

Um arquivo `script.js` com variáveis, funções, condicionais e laços aplicados ao contexto de produtos.

Ao final desta aula, seu projeto terá suas **primeiras lógicas em JavaScript** — a base para tornar o e-commerce verdadeiramente dinâmico.

Capacidades que Você Vai Desenvolver



Variáveis e Tipos

Armazenar e manipular dados como nome, preço e estoque de produtos usando `let` e `const`.



Operadores

Realizar cálculos aritméticos, comparações e operações lógicas essenciais para qualquer loja virtual.



Condicionais e Laços

Controlar o fluxo do programa: exibir promoção se houver desconto, listar todos os produtos em estoque.



Funções Reutilizáveis

Organizar o código em blocos reutilizáveis que poderão ser chamados em qualquer parte do projeto.

Variáveis: `let` e `const`

Declarando dados do produto

Variáveis são contêineres para armazenar valores. Em JavaScript moderno, usamos `let` para valores que **podem mudar** e `const` para valores **fixos**.

```
const nomeLoja = "Minha Loja Online";  
  
let produto1 = "Smartphone";  
let preco1 = 1500;  
let emPromocao = true;
```

Tipos de dados

String

Texto: "Smartphone"

Number

Número: 1500

Boolean

Lógico: true

Null

Vazio: null

Undefined

Indefinido

❏ Use sempre `const` como padrão. Só troque para `let` quando souber que o valor precisará mudar.

Operadores em Ação

Aritméticos

```
const preco = 299.90;
const desconto = 0.15; // 15%

// Calculando o preço final
const valorDesconto = preco * desconto;
// 44.985

const precoFinal = preco - valorDesconto;
// 254.915

const frete = 19.90;
const total = precoFinal + frete;
// 274.815
```

```
function calcularDesconto(preco, desconto) {
  return preco - (preco * desconto);}
```

Relacionais e Lógicos

```
const estoque = 15;
const emPromocao = true;
const precoFinal = 254.90;

// Relacionais: ==, !=, >, <, >=, <=
estoque > 0 // true
precoFinal <= 300 // true

// Lógicos: && (e), || (ou), ! (não)
estoque > 0 && emPromocao // true
estoque === 0 || emPromocao // true
!emPromocao // false
```

 Prefira sempre `===` (igualdade estrita) em vez de `==` — ela compara valor e tipo.

Template Strings: Textos Dinâmicos

Template strings usam **backticks** (```) e permitem inserir variáveis diretamente no texto com `${variavel}` — muito mais legível do que concatenação!

✗ Forma antiga (concatenação)

```
const nome = "Tênis Running Pro";
const preco = 299.90;
const estoque = 15;

const msg = "Produto: " + nome +
  " | Preço: R$ " + preco +
  " | Estoque: " + estoque + " un.";
```

```
console.log(`Produto:${produto1}`);
console.log(`Preço final: R$ ${precoFinal}`);
```

✓ Com Template String

```
const nome = "Tênis Running Pro";
const preco = 299.90;
const estoque = 15;

const msg = `Produto: ${nome}
Preço: R$ ${preco.toFixed(2)}
Estoque: ${estoque} unidades`;

console.log(msg);
```

☑ Template strings também suportam múltiplas linhas e expressões como `${preco * 1.1}` diretamente dentro das chaves.

Estruturas Condicionais

if / else — Promoção do produto

```
let precoFinal;  
  if (emPromocao) {  
    precoFinal = calcularDesconto(preco1, 0.1);  
  } else {  
    precoFinal = preco1;  
  }
```

Laços de Repetição

for — Listando produtos

```
const produtos = [  
  "Tênis Running Pro",  
  "Mochila Sport",  
  "Camiseta Dry-Fit",  
  "Meias Esportivas"  
];  
  
for (let i = 0; i < produtos.length; i++) {  
  console.log(`${i + 1}. ${produtos[i]}`);  
}  
  
// Saída:  
// 1. Tênis Running Pro  
// 2. Mochila Sport  
// 3. Camiseta Dry-Fit  
// 4. Meias Esportivas
```

while — Controlando estoque

```
let estoque = 5;  
let vendas = 0;  
  
while (estoque > 0) {  
  console.log(  
    `Venda realizada!  
    Estoque restante: ${estoque - 1}`  
  );  
  estoque--;  
  vendas++;  
}  
  
console.log(  
  `Total de vendas: ${vendas}`  
);  
// Total de vendas: 5
```

⚠ Cuidado com **loops infinitos**! Sempre garanta que a condição do `while` eventualmente se torne falsa.

Laços de Repetição

for — Listando produtos

```
const produtos = [  
  { nome: "Smartphone", preco: 1500 },  
  { nome: "Camiseta", preco: 80 },  
  { nome: "Relógio", preco: 250 }  
];  
  
for (let i = 0; i < produtos.length; i++) {  
  console.log(  
    `Produto: ${produtos[i].nome} - R$ ${produtos[i].preco}`  
  );  
}
```

```
function () {
```



? !

```
}
```

Funções: O Coração da Reutilização

Funções são blocos de código que executam uma tarefa específica e podem ser **chamados quantas vezes forem necessários**. São fundamentais para organizar o código do e-commerce.

Três Formas de Declarar Funções

Declaração de Função

```
function calcularDesconto(preco, percentual)
{
  const desconto = preco * (percentual /
100);
  return preco - desconto;
}

const final = calcularDesconto(299.90, 15);
console.log(`R$ ${final.toFixed(2)}`); // R$
254.92
```

Pode ser chamada **antes** de ser declarada no código (hoisting).

Expressão de Função

```
const exibirProduto = function(nome, preco) {
  console.log(`Produto: ${nome}
Preço: R$ ${preco.toFixed(2)}`);
};
```

```
const exibirProduto = (produto) => {
  return `Produto: ${produto.nome} - R$ ${produto.preco}`;
};
console.log(exibirProduto(produtos[0]));
```

Parâmetros, Retorno e Valores Padrão

Função

```
// Parâmetro com valor padrão: desconto = 0
function calcularPrecoFinal(
  preco,
  desconto = 0,
  frete = 0
) {
  const valorDesconto = preco * (desconto / 100);
  const precoComDesconto = preco - valorDesconto;
  const total = precoComDesconto + frete;

  return {
    original: preco,
    desconto: valorDesconto,
    frete: frete,
    total: total
  };
}

const resultado = calcularPrecoFinal(299.90, 15,
19.90);
console.log(resultado.total.toFixed(2)); // 274.82
```

Por que usar `return`?

O `return` encerra a função e **devolve um valor** para quem a chamou. Sem ele, a função executa ações mas não entrega resultado.

Com return

Você **recebe** o valor calculado para usar em outras partes do código.

Sem return

A função apenas **executa** ações (como exibir no console) mas não entrega dados.

```
function calcularDesconto(preco, desconto) {
  return preco - (preco * desconto);
}
```



Atividade Prática

MÃO NA MASSA

Chegou a hora de colocar em prática tudo que aprendemos! Você vai criar o arquivo `script.js` e conectá-lo ao projeto do e-commerce.

01

Criar o arquivo

Crie `script.js` na raiz do projeto e adicione a tag `<script src="script.js">` antes de fechar o `</body>` no HTML.

02

Declarar variáveis de produtos

Use `const` e `let` para representar pelo menos 3 produtos com nome, preço, categoria e estoque.

03

Criar funções

Escreva funções para exibir informações do produto e calcular desconto. Use template strings nas mensagens.

04

Usar condicionais e laços

Verifique se o produto está em promoção com `if/else`. Liste todos os produtos com um laço `for`.

05

Testar no console

Abra o DevTools (F12) no navegador e verifique os resultados no console. Corrija possíveis erros.



Passo 1: Conectando o JavaScript ao HTML

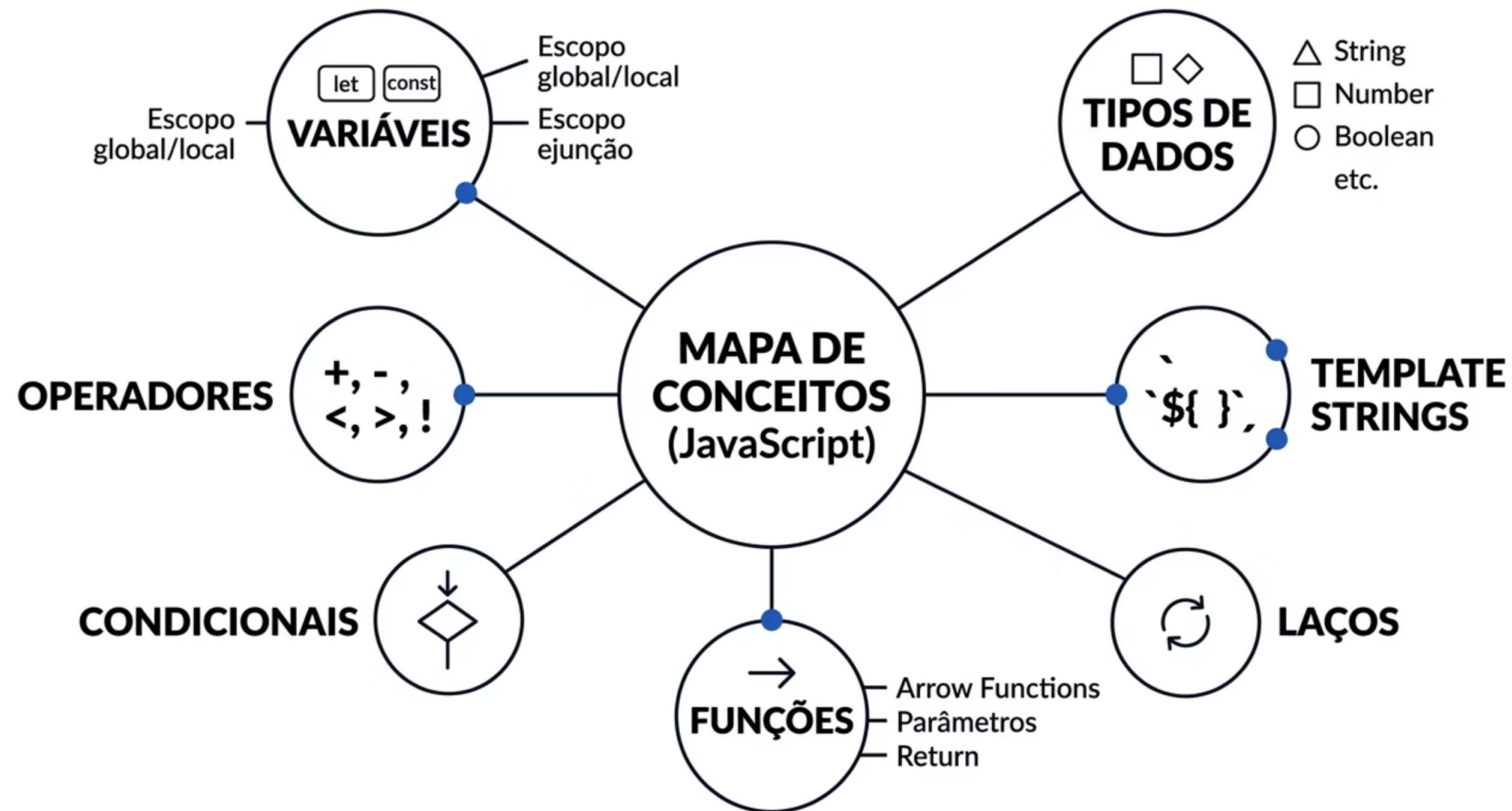
Antes de escrever qualquer lógica, precisamos dizer ao HTML onde está nosso arquivo JavaScript. A tag `<script>` deve ser inserida **no final do body**, garantindo que o HTML carregue antes da execução do script.

```
<!-- index.html – Estrutura básica do e-commerce -->
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TechShop – E-commerce</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <!-- Todo o conteúdo HTML do e-commerce aqui -->
  <header>...</header>
  <main>...</main>
  <footer>...</footer>

  <!-- Script sempre ao final do body -->
  <script src="script.js"></script>
</body>
```

Resumo dos Conceitos Aprendidos



Todos esses conceitos trabalharam juntos para criar a lógica do nosso e-commerce — da declaração de variáveis até a listagem de produtos com desconto.

Comparando as Formas de Função

Característica	Declaração	Expressão	Arrow Function
Sintaxe	<code>function nome(){} </code>	<code>const f = function(){} </code>	<code>const f = () => {} </code>
Hoisting	✅ Sim	❌ Não	❌ Não
Uso ideal	Funções principais	Callbacks simples	Funções curtas, callbacks
No e-commerce	<code>calcularDesconto()</code>	<code>exibirProduto()</code>	<code>formatarPreco()</code>

📘 Para iniciantes, comece com a declaração de função — é a mais explícita. Aos poucos, adote arrow functions para código mais conciso e moderno.

Erros Comuns – Fique Atento!

⚠ Usar `var` em vez de `let/const`

`var` tem escopo de função (não de bloco) e causa comportamentos inesperados. Sempre prefira `let` e `const` no código moderno.

⚠ Loop infinito no `while`

Se a condição nunca se tornar falsa, o navegador trava. Sempre garanta que a variável de controle é atualizada dentro do loop.

⚠ Esquecer o `break` no `switch`

Sem `break`, o `switch` continua executando os próximos casos (fall-through). Sempre adicione `break` ao final de cada `case`.

⚠ Usar `==` em vez de `===`

`"5" == 5` retorna `true` com `==` (converte tipos). Use sempre `===` para comparações seguras sem conversão automática.

